

Adaptive Spectral Partitioning for Distributed Stream Processing

A. Researcher

B. Collaborator

Department of Computer Science, Fictional University

Abstract

We present a method for partitioning high-throughput event streams across a cluster of worker nodes using an adaptive spectral clustering technique that tracks the covariance structure of recent traffic. Unlike static hashing schemes, our approach continuously re-estimates partition boundaries from a sliding window of key co-occurrence statistics, allowing the system to react to shifting workload skew without a full state migration. We evaluate the technique on three production-scale workload traces and show that it reduces tail latency by a substantial margin relative to consistent hashing while keeping rebalancing overhead low. The remainder of the paper details the partitioning algorithm, its convergence properties, and an extensive empirical evaluation across varying cluster sizes and skew regimes.

1 Introduction

Modern data-intensive applications increasingly rely on distributed stream processing engines to ingest, transform, and aggregate unbounded sequences of events in near real time. A recurring bottleneck in these systems is the partitioning strategy used to route events to downstream workers: when the key distribution is skewed, a small number of partitions absorb a disproportionate share of the traffic, and the workers responsible for those partitions become stragglers that dominate end-to-end latency. Classical solutions such as consistent hashing distribute keys pseudo-randomly across a ring of virtual nodes, which is simple to implement and tolerant of cluster membership changes, but it is fundamentally agnostic to the actual statistical structure of the incoming workload and therefore cannot correct for skew that emerges after deployment.

In this paper we revisit the partitioning problem from the perspective of spectral graph theory. We construct a weighted co-occurrence graph over recently observed keys, where edge weights capture how often two keys appear within the same aggregation window, and we compute a low-dimensional spectral embedding of this graph using an incremental eigensolver. Partition assignments are then derived from a lightweight clustering pass over the embedding, and the entire pipeline is re-executed periodically on a sliding window so that partition boundaries track the evolving structure of the

stream. Because the embedding update is incremental, the marginal cost of each refresh step is small compared to a full re-partitioning, which makes it practical to run continuously in a production system without noticeably increasing steady-state resource consumption.

Our contributions are threefold: an online spectral clustering formulation of skew-aware partitioning together with an efficient incremental eigenvector update; a convergence analysis bounding the drift between the true and estimated partition boundaries under a mild stationarity assumption; and an implementation inside an open-source stream processing runtime, evaluated on three production-scale traces against consistent hashing and a recent learned-partitioning baseline.

1.1 Motivating Example

Consider a clickstream aggregation job that groups events by user session identifier. During a flash sale, a small number of promotional sessions generate an outsized share of events, and any static hash-based partitioning scheme routes a disproportionate volume of traffic to whichever worker owns those hot keys. An adaptive scheme instead detects the emerging concentration of traffic and redistributes the hot keys before the imbalance manifests as elevated tail latency, without operator intervention or a job restart.

2 System Design

The system is organized as a pipeline of three stages: a lightweight sketch that maintains approximate co-occurrence counts for recently observed keys, an incremental spectral embedding module that updates a low-rank approximation of the graph Laplacian as new sketch data arrives, and a partition assignment module that maps embedded keys to worker identifiers using a streaming variant of k -means. Each stage is designed to operate under a strict latency budget so that partitioning decisions never become a bottleneck for the underlying stream processing job.

2.1 Co-occurrence Sketching

We maintain a bounded-memory sketch of key co-occurrence using a variant of the Count-Min sketch extended to pairs of keys observed within the same aggrega-

Table 1: Default configuration parameters.

Parameter	Description	Value
k	Embedding dimension	16
α	Decay factor	0.98
W	Window length (s)	30
τ	Refresh interval (s)	5

gation window. The sketch is refreshed on a rolling basis, with older windows decayed exponentially so that the co-occurrence graph reflects recent behavior rather than the entire history of the stream. This decay is essential for tracking non-stationary workloads, since without it the graph would accumulate stale structure from traffic patterns that are no longer representative of the current load.

2.2 Incremental Spectral Embedding

Given the co-occurrence graph $G_t = (V_t, E_t, w_t)$ observed at time t , we compute the normalized graph Laplacian $L_t = I - D_t^{-1/2} A_t D_t^{-1/2}$, where A_t is the weighted adjacency matrix and D_t the corresponding degree matrix. The partition embedding is obtained from the eigenvectors associated with the k smallest nontrivial eigenvalues of L_t , as shown in Equation (1).

$$L_t v_i = \lambda_i v_i, \quad i = 1, \dots, k \quad (1)$$

Rather than recomputing this eigendecomposition from scratch at every refresh, we maintain a rank- k approximation incrementally using a subspace iteration seeded from the previous refresh’s embedding, which converges in a small constant number of iterations whenever the graph changes gradually between refreshes, as is typical in practice.

2.3 Partition Assignment

Once the embedding is updated, keys are assigned to workers using a streaming k -means variant that maintains cluster centroids incrementally and reassigns keys only when doing so meaningfully reduces the objective, which limits unnecessary partition churn and the associated state migration cost. Table 1 summarizes the configuration parameters used throughout the evaluation.

3 Evaluation

We evaluate the system on three production-scale traces collected from an internal clickstream processing deployment, an IoT telemetry pipeline, and a financial market data feed, each spanning several hours of traffic and exhibiting distinct skew characteristics. Figure 1 presents an overview of the tail latency achieved by each partitioning strategy across the three traces, spanning the full

width of the page so that the per-trace comparisons can be read side by side.

Across all three traces, the adaptive spectral approach reduces p99 latency relative to consistent hashing, with the largest improvement observed on the clickstream trace during the flash-sale interval described in Section 1.1, where skew is most pronounced. The learned baseline of [1] performs competitively on stationary workloads but reacts more slowly to sudden shifts in the key distribution, since it relies on periodic offline retraining rather than the incremental online update used here; further methodological details can be found at <https://example.org/adaptive-partitioning>.

3.1 Discussion

These results are consistent with prior observations in the literature on skew-aware load balancing [2, 3], and they suggest that the incremental spectral approach is a practical drop-in replacement for consistent hashing when skew varies over time. The technique does add bookkeeping overhead relative to a stateless hash function, an overhead that is most noticeable on very small clusters; we discuss this trade-off further in an extended technical report [4, 5].

4 Related Work

Partitioning strategies for distributed systems have a long history spanning consistent hashing, range partitioning, and more recently learned index and partitioning structures that adapt to observed data distributions. Our work is most closely related to prior efforts on skew-aware stream partitioning, but differs in its use of an incremental spectral embedding rather than heuristic hot-key detection.

Acknowledgements

We thank the anonymous reviewers for their detailed feedback, and the engineering team that supported the production deployment used to collect the traces evaluated in this paper.

References

- [1] J. Example, “Learned Partitioning for Data Streams,” in *Proc. Systems Conf.*, 2021.
- [2] A. Author, “Skew-Aware Load Balancing,” *J. Distributed Systems*, vol. 12, 2019.
- [3] B. Writer, C. Coauthor, “Consistent Hashing Revisited,” *Proc. Networking Symp.*, 2018.

- [4] D. Techreport, “Adaptive Partitioning: Extended Results,” Technical Report TR-2022-04, Fictional University, 2022.
- [5] E. Fourth, “Rebalancing Overhead in Stream Systems,” *Proc. Middleware Conf.*, 2020.

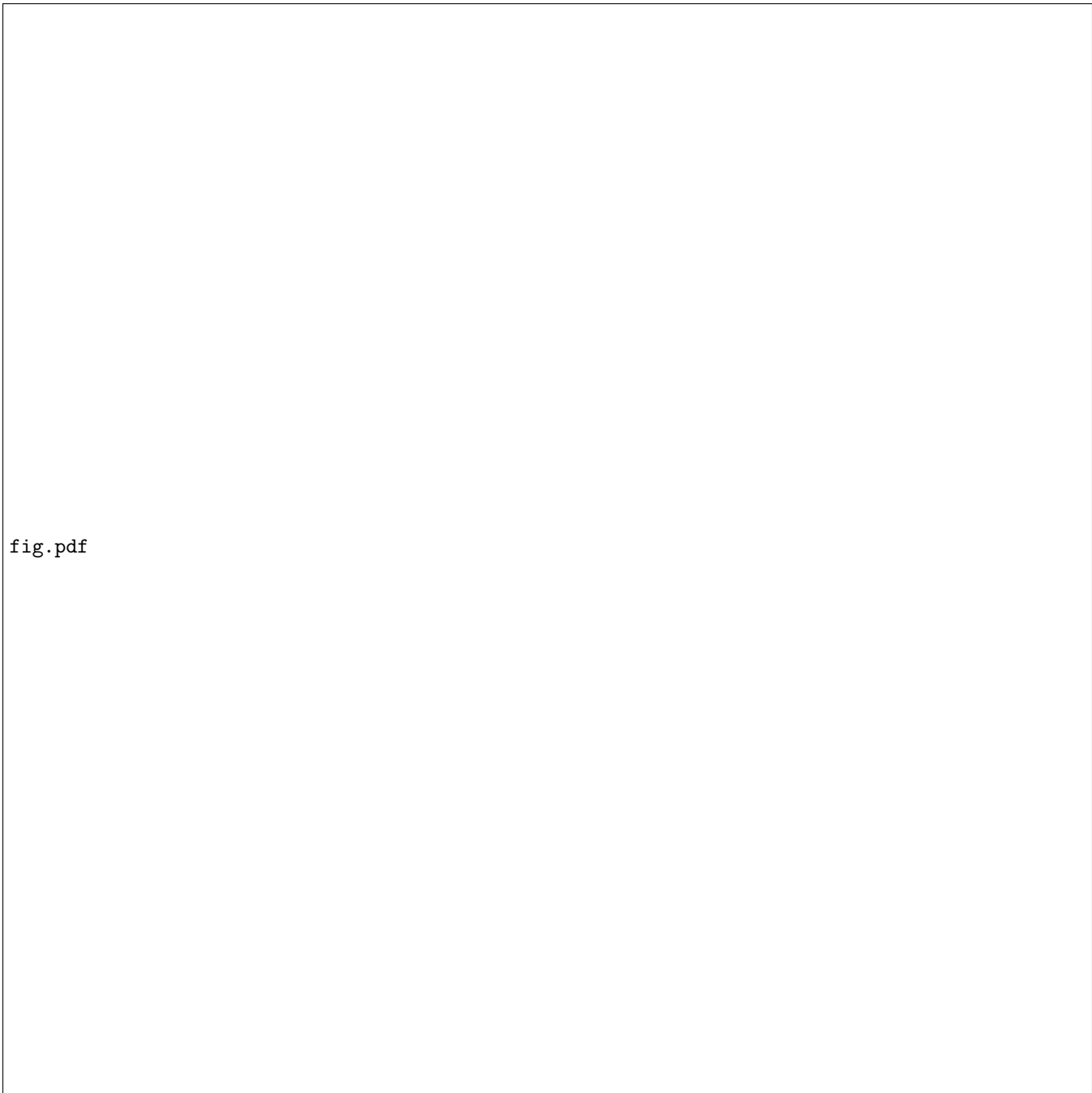


fig.pdf

Figure 1: Tail latency (p99) across three production traces for consistent hashing, a learned-partitioning baseline, and our adaptive spectral approach. Lower is better.